

1. What is the purpose of Exception Handling in Java?

Answer:

The purpose of exception handling is to gracefully manage runtime errors so that the normal flow of the application can be maintained. It prevents abrupt program termination and allows developers to respond to errors (e.g., showing user-friendly messages or logging).

Example:

```
java

try {
    int result = 10 / 0; // ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero!");
}
```

2. Differentiate between checked and unchecked exceptions.

Feature	Checked Exception	Unchecked Exception
Checked at	Compile-time	Runtime
Requirement	Must be caught or declared using throws	Optional to handle (no compilation error)
Parent class	Exception (excluding RuntimeException)	RuntimeException
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

Example:

```
java

// Checked - must handle
try {
    FileReader file = new FileReader("test.txt");
} catch (IOException e) { }
```



```
// Unchecked - optional to handle
int result = 10 / 0; // May crash but compiles fine
```

3. Differentiate between throw and throws in Exception Handling.

Feature	throw	throws
Purpose	Manually throw an exception object	Declare that a method may throw exceptions
Location	Inside method body	In method signature
Usage	Used with single exception instance	Can declare multiple exceptions (comma-separated)
Execution	Actually throws the exception at runtime	Compile-time declaration for caller handling

Example:

java

```
// throw - manual throwing
public void validateAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException("Underage");
    }
}

// throws - declaring possible exceptions
public void readFile(String path) throws IOException, FileNotFoundException {
    // method body
}
```